The OpenMP:

Runtime Library and Environment Variables

OpenMP: Runtime Library and Environment Variables

The **runtime library** is a collection of predefined functions that help the programmer **control**, **query**, and **manage** OpenMP during execution.

The runtime library mainly provides functions for:

- Thread management
- Processor information
- Timing
- Locks
- Nested parallelism
- Dynamic threads
- Scheduling

OpenMP: Runtime Library and Environment Variables

Thread Management Functions

omp_set_num_threads(): Sets the number of threads for future parallel regions.

Syntax: omp_set_num_threads(int n);

Example:

```
#include <omp.h>
#include <stdio.h>

int main()
{
    omp_set_num_threads(4);

#pragma omp parallel
    {
        printf("Hello\n");
    }
}
```

Output:

```
Hello
Hello
Hello
Hello
```

OpenMP: Runtime Library and Environment Variables

Thread Management Functions

omp_get_num_threads(): Returns the number of threads in the current team.

```
#pragma omp parallel
{
    printf("%d\n", omp_get_num_threads());
}
```

Output:

```
4
4
4
4
```

omp_get_thread_num(): Returns the ID of the current thread.

```
#pragma omp parallel
{
    printf("Thread %d\n", omp_get_thread_num());
}
```

Output:

```
Thread 0
Thread 1
Thread 2
Thread 3
```

OpenMP: Runtime Library and Environment Variables

Thread Management Functions

omp_get_max_threads(): Returns the maximum number of threads OpenMP may use.

```
printf("%d", omp_get_max_threads());
```

Output:

```
8
```

omp_in_parallel(): Checks whether execution is inside a parallel region. Returns: 1 → Yes or 0 → No.

```
printf("%d\n", omp_in_parallel());

#pragma omp parallel
{
    printf("%d\n", omp_in_parallel());
}
```

Output:

```
0
1
1
1
1
```

OpenMP: Runtime Library and Environment Variables

Timing Functions

omp_get_wtime(): Returns the wall-clock time.

```
double start = omp_get_wtime();

/* computation */

double end = omp_get_wtime();

printf("Time = %f", end-start);
```

wall clock = actual real clock

Output:

Time = 0.523 seconds

omp_get_wtick(): Returns the timer resolution.

```
printf("%f", omp_get_wtick());
```

Output:
0.000001

Resolution = The smallest time difference that the timer can reliably measure.

OpenMP: Runtime Library and Environment Variables

Processor Information

omp_get_num_procs(): Returns the number of available processors.

```
printf("%d", omp_get_num_procs());
```

Output: **8**

Dynamic Thread Control

Normally: Program ⟹ Uses fixed number of threads

Sometimes OpenMP decides fewer threads are better.

Enable

```
omp_set_dynamic(1);
```

Check status

```
omp_get_dynamic();
```

if dynamic enable - can change automatically while the program runs.

OpenMP: Runtime Library and Environment Variables

Nested Parallelism

Normally: Parallel ⟹ Another Parallel

The second parallel region is ignored unless nested parallelism is enabled.

Enable: `omp_set_nested(1);`

enable nested allow also make parallel processing inside a nested process

Locks

Locks: Locks prevent multiple threads from modifying shared data simultaneously.

- omp_init_lock()
- omp_set_lock()
- omp_unset_lock()
- omp_destroy_lock()

```
omp_lock_t lock;
omp_init_lock(&lock);
#pragma omp parallel
{
    omp_set_lock(&lock);
    printf("Critical Work\n");
    omp_unset_lock(&lock);
}
omp_destroy_lock(&lock);
```

Only one thread enters the locked section at a time.

OpenMP: Runtime Library and Environment Variables

Environment Variables

Instead of changing the program, we can configure OpenMP before running it using **environment variables**.

Without recompiling, we can change:

- number of threads
- scheduling policy
- stack size
- processor binding
- dynamic threading

just by changing environment variables.

OpenMP: Runtime Library and Environment Variables

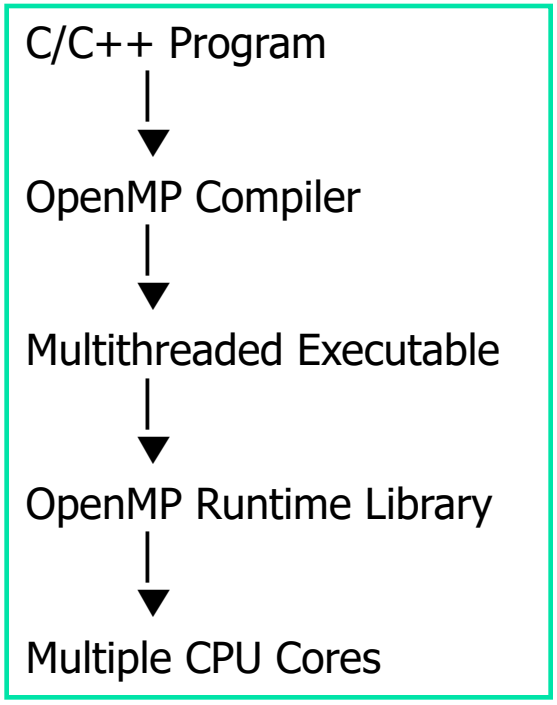
Common OpenMP Environment Variables

Variable	Purpose
OMP_NUM_THREADS	Number of threads
OMP_DYNAMIC	Enable/disable dynamic threads
OMP_SCHEDULE	Scheduling method
OMP_PROC_BIND	Bind threads to processors
OMP_PLACES	Define where threads execute
OMP_STACKSIZE	Stack size per thread
OMP_WAIT_POLICY	Active or passive waiting
OMP_MAX_ACTIVE_LEVELS	Maximum nested parallel levels

OpenMP: The OpenMP Compiler

An OpenMP compiler is a compiler capable of translating OpenMP directives into executable multithreaded machine code using the OpenMP runtime library.

Role of the OpenMP Compiler



What Does the Compiler Do?

Step 1: Reads Source Code

Step 2: Detects OpenMP Directives (commands)

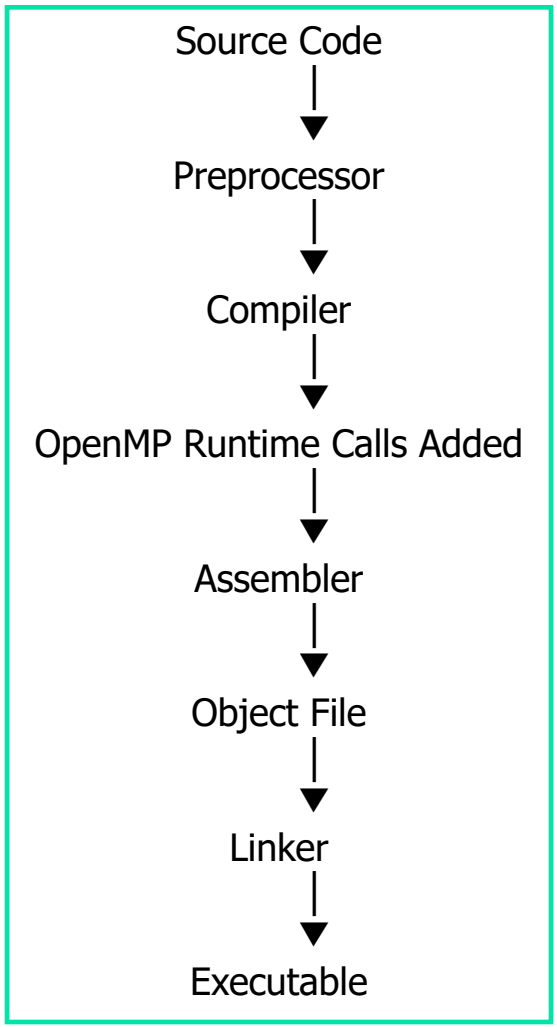
Step 3: Generates Thread Creation Code

Step 4: Links Runtime Library

Step 5: Produces Executable

OpenMP: The OpenMP Compiler

OpenMP Compilation Process



Without OpenMP Support

```
#pragma omp parallel
{
    printf("Hello\n");
}
```

Output: Hello

With OpenMP Support

```
#pragma omp parallel
{
    printf("Hello\n");
}
```

Output:

Hello
Hello
Hello
Hello

OpenMP: The OpenMP Compiler

OpenMP Compiler Directives

The compiler recognizes directives beginning with **#pragma omp**

#pragma omp parallel

#pragma omp for

#pragma omp sections

#pragma omp single

Compiler Ignores Unknown Pragmas

OpenMP: The OpenMP Compiler

Popular OpenMP Compilers

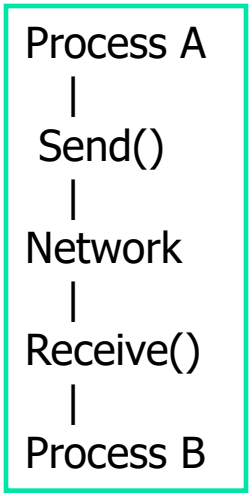
Compiler	Supports OpenMP	Operating System
GCC (g++)	Yes	Linux, Windows, macOS
Clang	Yes	Linux, macOS, Windows
Microsoft Visual C++ (MSVC)	Yes	Windows
Intel oneAPI DPC++/C++ Compiler (icx/icpx)	Yes	Windows, Linux
NVIDIA HPC SDK (nvc/nvc++)	Yes	Linux, Windows

The MPI: Message Passing Interface

Message Passing: Message Passing is a communication method where processes exchange information by sending and receiving messages.

Each process:

- has its own private memory
- cannot directly access another process's memory
- communicates through messages



The MPI: Message Passing Interface

Why Message Passing?

Shared memory systems have limitations.

- Memory contention

Memory contention = multiple processors competing for the same memory resources to update or access.

- Synchronization overhead
- Limited scalability

Distributed memory systems solve these issues.

- Highly scalable
- Better fault isolation
- Suitable for supercomputers
- Efficient for large scientific applications

The MPI: Message Passing Interface

MPI is

- an international standard
- a programming library
- designed for distributed-memory parallel computing

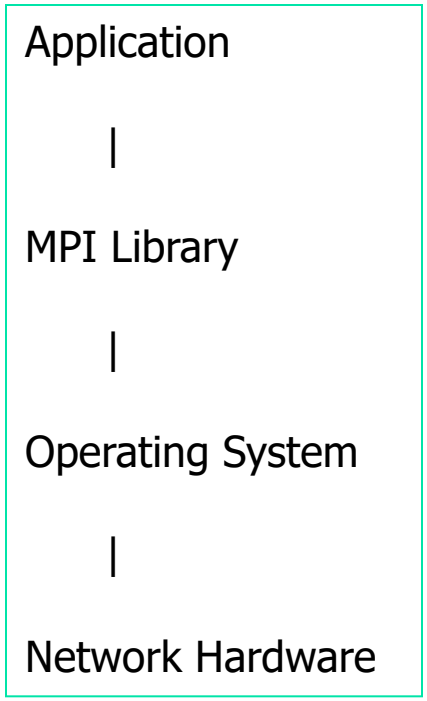
MPI is **not a programming language**.

It is a library used with

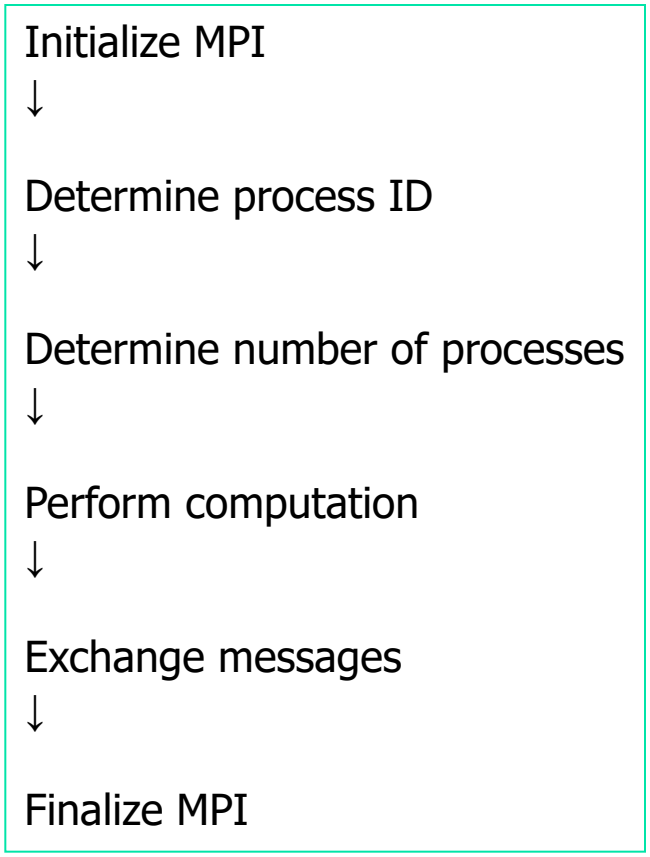
- C
- C++
- Fortran

The MPI: Message Passing Interface

MPI Architecture



MPI Program Structure



The MPI: Message Passing Interface

MPI Function	Purpose	Syntax	Example
MPI_Init()	Initializes the MPI execution environment. Must be the first MPI function called.	MPI_Init(&argc, &argv);	MPI_Init(&argc, &argv);
MPI_Finalize()	Terminates the MPI environment. Must be the last MPI function called.	MPI_Finalize();	MPI_Finalize();
MPI_Comm_size()	Returns the total number of processes in a communicator.	MPI_Comm_size(MPI_COMM_WORLD, &size);	If run with mpirun -np 4, then size = 4.
MPI_Comm_rank()	Returns the rank (ID) of the calling process.	MPI_Comm_rank(MPI_COMM_WORLD, &rank);	Process IDs are 0, 1, 2, 3.
MPI_Get_processor_name()	Returns the processor (host) name where the process is running.	MPI_Get_processor_name(name, &len);	char name[MPI_MAX_PROCESSOR_NAME]; MPI_Get_processor_name(name, &len);
MPI_Send()	Sends data from one process to another (blocking).	MPI_Send(buf, count, datatype, dest, tag, MPI_COMM_WORLD);	MPI_Send(&value, 1, MPI_INT, 1, 0, MPI_COMM_WORLD);
MPI_Recv()	Receives data sent by another process (blocking).	MPI_Recv(buf, count, datatype, source, tag, MPI_COMM_WORLD, MPI_STATUS_IGNORE);	MPI_Recv(&value, 1, MPI_INT, 0, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
MPI_Isend()	Sends data without waiting for completion (non-blocking).	MPI_Isend(buf, count, datatype, dest, tag, MPI_COMM_WORLD, &request);	MPI_Isend(&value, 1, MPI_INT, 1, 0, MPI_COMM_WORLD, &req);

The MPI: Message Passing Interface

MPI Function	Purpose	Syntax	Example
MPI_Irecv()	Receives data without blocking execution.	MPI_Irecv(buf, count, datatype, source, tag, MPI_COMM_WORLD, &request);	MPI_Irecv(&value, 1, MPI_INT, 0, 0, MPI_COMM_WORLD, &req);
MPI_Wait()	Waits until a non-blocking communication completes.	MPI_Wait(&request, MPI_STATUS_IGNORE);	MPI_Wait(&req, MPI_STATUS_IGNORE);
MPI_Barrier()	Synchronizes all processes in a communicator.	MPI_Barrier(MPI_COMM_WORLD);	All processes wait until every process reaches the barrier.
MPI_Bcast()	Broadcasts data from one process to all other processes.	MPI_Bcast(buf, count, datatype, root, MPI_COMM_WORLD);	MPI_Bcast(&value, 1, MPI_INT, 0, MPI_COMM_WORLD);
MPI_Scatter()	Distributes equal portions of an array from one process to all processes.	MPI_Scatter(sendbuf, sendcount, datatype, recvbuf, recvcount, datatype, root, MPI_COMM_WORLD);	Root sends one integer to each process.
MPI_Gather()	Collects data from all processes into one process.	MPI_Gather(sendbuf, sendcount, datatype, recvbuf, recvcount, datatype, root, MPI_COMM_WORLD);	Each process sends one integer to Rank 0.
MPI_Allgather()	Gathers data from all processes and distributes the complete result to every process.	MPI_Allgather(sendbuf, sendcount, datatype, recvbuf, recvcount, datatype, MPI_COMM_WORLD);	Every process receives all values.
MPI_Reduce()	Combines values from all processes using an operation (SUM, MAX, MIN, etc.) and stores the result at the root process.	MPI_Reduce(sendbuf, recvbuf, count, datatype, op, root, MPI_COMM_WORLD);	MPI_Reduce(&local, &sum, 1, MPI_INT, MPI_SUM, 0, MPI_COMM_WORLD);

The MPI: Message Passing Interface

MPI Function	Purpose	Syntax	Example
MPI_Allreduce()	Performs a reduction and distributes the result to all processes.	MPI_Allreduce(sendbuf, recvbuf, count, datatype, op, MPI_COMM_WORLD);	Every process gets the total sum.
MPI_Scan()	Performs a prefix reduction (partial reduction).	MPI_Scan(sendbuf, recvbuf, count, datatype, op, MPI_COMM_WORLD);	Computes cumulative sums across process ranks.
MPI_Probe()	Checks for an incoming message without receiving it.	MPI_Probe(source, tag, MPI_COMM_WORLD, &status);	Determines message size before receiving.
MPI_Get_count()	Returns the number of received elements in a message.	MPI_Get_count(&status, MPI_INT, &count);	Determines how many integers were received.
MPI_Wtime()	Returns the elapsed wall-clock time for performance measurement.	double t = MPI_Wtime();	double start = MPI_Wtime();
MPI_Abort()	Terminates all MPI processes immediately due to an error.	MPI_Abort(MPI_COMM_WORLD, errorcode);	MPI_Abort(MPI_COMM_WORLD, -1);

The MPI: Message Passing Interface

MPI Communicator: A communicator defines a communication group.

MPI_COMM_WORLD

Rank 0

Rank 1

Rank 2

Rank 3

Rank 4

Every process has a unique ID.

Usually: Rank 0 acts as

- Master
- Coordinator

Other ranks

- Workers

The MPI: Message Passing Interface

Basic MPI Hello World

```
#include <mpi.h>
#include <stdio.h>

int main(int argc, char *argv[])
{
    MPI_Init(&argc, &argv);

    int rank, size;

    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    printf("Hello from Process %d of %d\n", rank, size);

    MPI_Finalize();

    return 0;
}
```

Output

```
Hello from Process 0 of 4
Hello from Process 2 of 4
Hello from Process 1 of 4
Hello from Process 3 of 4
```

The MPI: Message Passing Interface

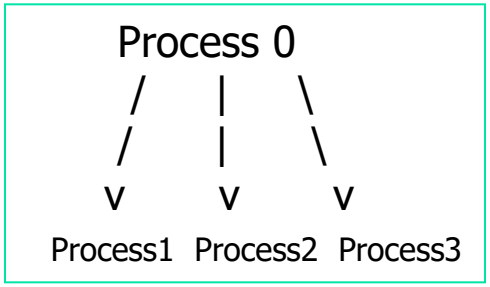
MPI Communication Types

A. Point-to-Point Communication: Communication occurs between **exactly two processes**.

- MPI_Send()
- MPI_Recv()

B. Collective Communication: Communication involves **all processes** within a communicator.

- MPI_Bcast()
- MPI_Scatter()
- MPI_Gather()
- MPI_Reduce()
- MPI_Allreduce()
- MPI_Barrier()



The MPI: Message Passing Interface

Point-to-Point Communication

MPI_Send(): Sends data to another process.

```
MPI_Send(
    buffer,
    count,
    datatype,
    destination,
    tag,
    MPI_COMM_WORLD
);
```

Parameter	Meaning
buffer	Data to send
count	Number of elements
datatype	MPI_INT, MPI_FLOAT, etc.
destination	Receiver rank
tag	Message identifier
communicator	Communication group

MPI_Recv(): Receives data.

```
MPI_Recv(
    buffer,
    count,
    datatype,
    source,
    tag,
    MPI_COMM_WORLD,
    MPI_STATUS_IGNORE
);
```

The MPI: Message Passing Interface

Example: Sending an Integer

```
#include <mpi.h>
#include <stdio.h>

int main(int argc, char *argv[])
{
    int rank, value;

    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);

    if(rank == 0)
    {
        value = 100;
        MPI_Send(&value, 1, MPI_INT, 1, 0, MPI_COMM_WORLD);
    }

    if(rank == 1)
    {
        MPI_Recv(&value, 1, MPI_INT, 0, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
        printf("Received value = %d\n", value);
    }

    MPI_Finalize();
}
```

Output

```
Process 0                Process 1
-----------              -----------
value = 100
MPI_Send        --------> MPI_Recv
                Prints: Received value = 100
```

The MPI: Message Passing Interface

Blocking vs Non-Blocking Communication

Blocking	Non-Blocking
Waits until operation completes	Returns immediately
Simpler to use	Allows overlap of communication and computation
Functions: MPI_Send, MPI_Recv	Functions: MPI_Isend, MPI_Irecv

Collective Communication Operations

Function	Purpose
MPI_Bcast()	Broadcast data from one process to all
MPI_Scatter()	Distribute portions of data to all processes
MPI_Gather()	Collect data from all processes to one
MPI_Reduce()	Combine values using operations such as sum, max, min
MPI_Allreduce()	Reduce and distribute the result to all processes
MPI_Barrier()	Synchronize all processes

The MPI: Message Passing Interface

Advantages of MPI

- Scalable to thousands of processors
- Portable across different platforms
- Efficient for distributed-memory systems
- Standardized API
- Supports both point-to-point and collective communication
- Widely used in High-Performance Computing (HPC)

Limitations of MPI

- Explicit communication increases programming complexity
- Debugging parallel programs can be difficult
- Communication overhead may affect performance
- Programmers must manage data distribution manually

The MPI: Message Passing Interface

MPI vs OpenMP

Feature	MPI	OpenMP
Memory Model	Distributed Memory	Shared Memory
Programming Style	Message Passing	Multithreading
Communication	Explicit (MPI_Send, MPI_Recv)	Implicit (shared variables)
Scalability	Excellent for clusters	Best for multicore shared-memory systems
Synchronization	Message-based and collective operations	Barriers, critical sections, locks
Typical Use	HPC clusters and supercomputers	Multi-core desktops and servers

The MPI: Message Passing Interface

Applications of MPI

- Weather forecasting and climate modeling
- Computational Fluid Dynamics (CFD)
- Finite Element Analysis (FEA)
- Molecular dynamics and bioinformatics
- Artificial Intelligence and distributed machine learning
- Big-data analytics
- Astrophysics simulations
- Seismic data processing